

Лабораторная работа «Статические и динамические переменные. Списки».

Тип памяти	Максимальный объем	
	х 86	х 64
Статическая	2 Гб	2 Гб
Стековая	1 Мб до 1 Гб	1 Мб до 1 Гб
Динамическая	2 Гб	8 терабайт

Статическая память.

```
cout << "char в байтах : " << sizeof(char) << endl;
cout << "short в байтах : " << sizeof(short) << endl;
cout << "int в байтах : " << sizeof(int) << endl;
cout << "long в байтах : " << sizeof(long) << endl;
cout << "float в байтах : " << sizeof(float) << endl;
cout << "double в байтах : " << sizeof(double) << endl;
```

Пример:

int r=1234; (int – 4 байта),

по адресу 00 41 66 90 (4 байта на адрес) находится значение 1234 (04D2₁₆)

адрес	данные
-------	--------

4 байта

4 байта

00 41 66 90

04D2

&r – получить адрес переменной r (адрес, по которому хранится значение 04D2)

```
cout << "    адрес r= " << &r << " " << endl;
```

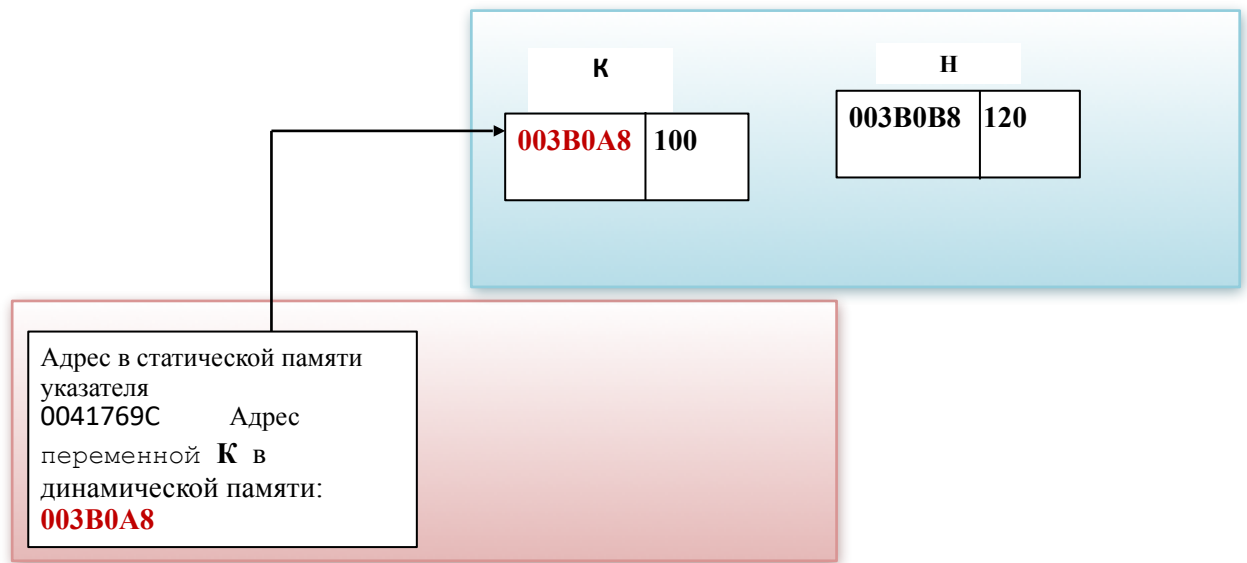
В соответствии с приведенным примером:

Вывод на экран адреса переменной r в статической памяти:

```
cout << " адрес r = " << &r << " " << endl;
```

Результат: **00416690.**

Динамическая память.



В с++ работа с динамической памятью:

- **new** — для выделения памяти;
- **delete** — для освобождения памяти.

Оператор **new** используется в следующих формах:

new тип; // для переменных

new тип[размер]; // для массивов

delete указатель; //для одного элемента

delete[] указатель; // для массива

delete указатель на первый элемент // для списка

ПРИМЕР 1.

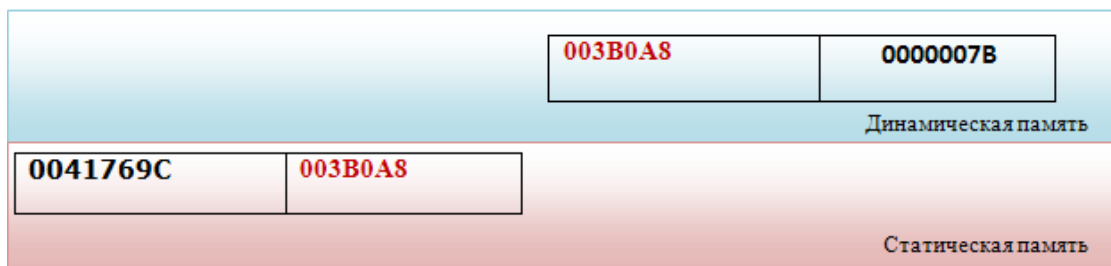
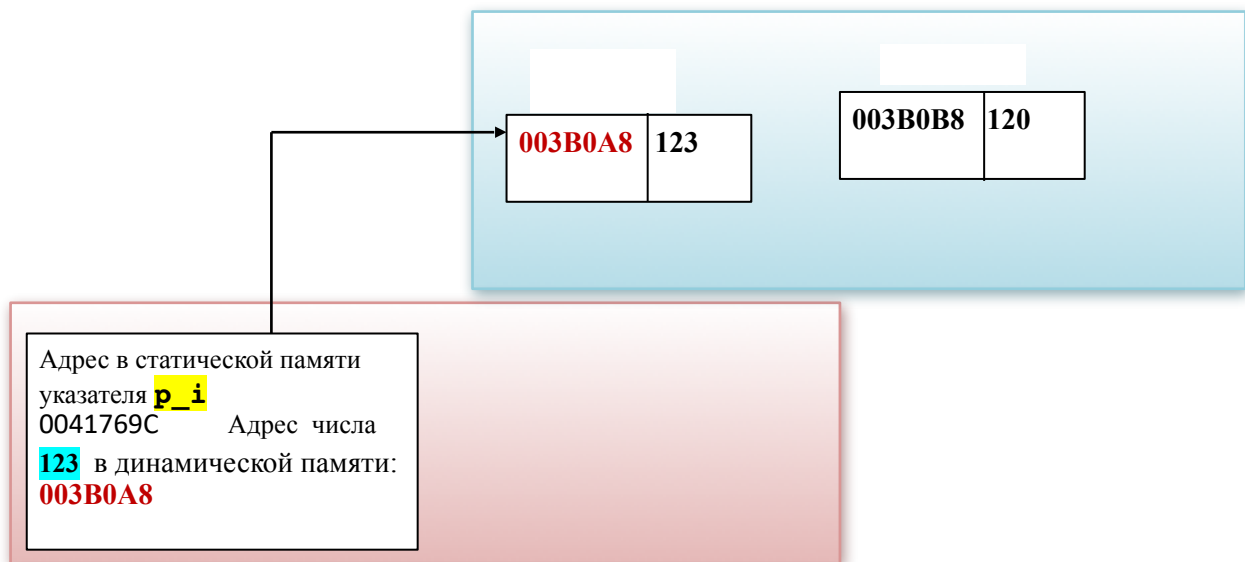
```
int *p_i, *p1_i; // объявление указателей p_i, и p1_i, указатель на ячейку памяти, где хранится int.  
double *p2_d;
```

```
p_i = new int; // выделена дин. память (адрес) для переменной типа int. (результат new -  
указатель на отведенную память)
```

```
p1_i = new int;
```

```
//операция * - операция разыменования.
```

```
*p_i=123; *p1_i=10; // обращение по адресу, который указывает p_i и p1_i(значение 123  
присваивается той переменной, адрес которой хранится в p_i)
```



В соответствии с приведенным примером:

- Вывод на экран адреса указателя **p_i** в статической памяти:

```
cout << " адрес указателя p_i " << &p_i << " " << endl;
```

Результат: адрес указателя **p_i** 0041769C

- Вывод на экран значения указателя **p_i** (адрес числа 123 в динамической памяти):

```
cout << " значения указателя p_i = " << p_i << " " << endl;
```

Результат: значения указателя **p_i** = 003B0A8

- Вывод на экран значения переменной, адрес которой хранится в `p_i`:

```
cout << "значение переменной = " << *p_i << " " << endl;
```

Результат: значение переменной = 123

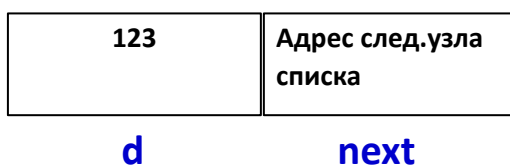
СПИСКИ

Линейные однонаправленные списки.

I. Однонаправленные списки без заглавного звена.

Связный список является простейшим типом данных динамической структуры, состоящей из элементов (**узлов**).

Пример узла списка:

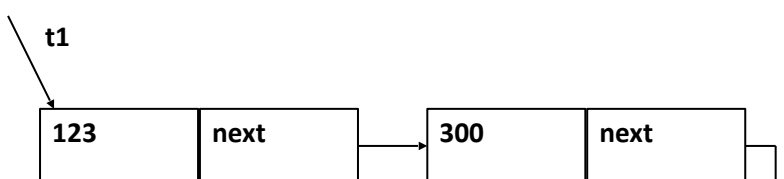


Описание узла списка

```
#include <iostream>
#include <stdlib.h>
#include <locale>
using namespace std;
```

```
struct n
{
    int d;    // - данные
    n *next; // - указатель на следующий адрес
};
... ..
int main()
{
    ... ..
}
```

Графическая интерпретация списка: **t1** указывает на то что ПЕРЕД 123, а не на сам 123; след стрелочка на левый верхний угол ячейки с 300



Основные операции над списками:

1. вставка узла (звена):

1. в начало списка;
2. в конец списка;
3. в произвольное место списка, например, после узла с указанным значением.

2. Просмотр элементов списка (вывод).

3. удаление узла:

- 2.1. из начала списка;
- 2.2. с конца списка;
- 2.3. удаление звена из произвольного места списка, например, удаление узла с указанным значением

4. Поиск узла с заданным значением

1. Вставка узла

1.1. Вставка в начало списка

• СОЗДАНИЕ УЗЛА СПИСКА

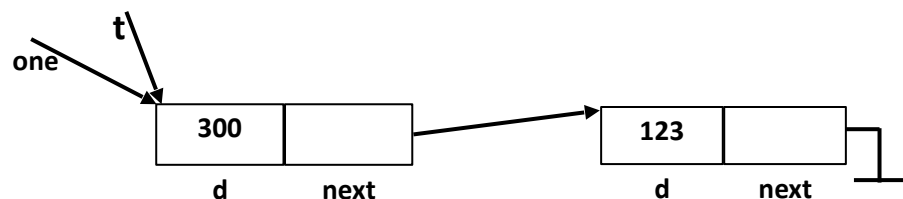
<pre>n *one=NULL; // указатели t и one для структуры узла списка (n) n *t=NULL;</pre>	<pre>struct n { int d; // - данные n *next; // - указатель на следующий адрес };</pre> Указатель one всегда указывает на первый элемент списка
<pre>t = new (n);</pre>	Начать рисовать прямоугол. С t и next
<pre>(*t).next=NULL;</pre>	«NULL – заземлено»
<pre>(*t).d = 123;</pre>	Поле d и значение 123

one=t;	Перевод пред адреса на данный (адрес d)
---------------	---

• ВСТАВКА УЗЛА СО ЗНАЧЕНИЕМ 123 В НАЧАЛО СПИСКА

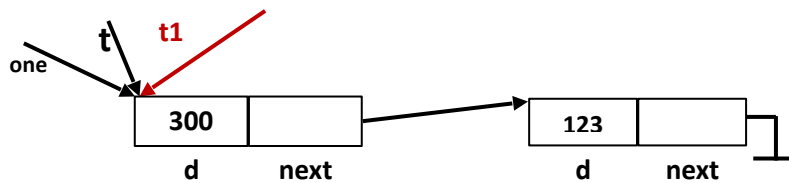
Есть один элемент списка	
Создание нового узла списка со значением 300 t = new (n); (*t).next=NULL; (*t).d = 300;	Указатель t - «временный» для манипулировали адресами, создание нового компонента(элемента)
Соединение элементов (перевод указателей) (*t).next=one;	
one=t;	

создан список из двух элементов:



Просмотр элементов списка

```
n *t1;
for (t1=one; t1!=NULL; t1=(*t1).next) cout<<" "<< (*t1).d;
```



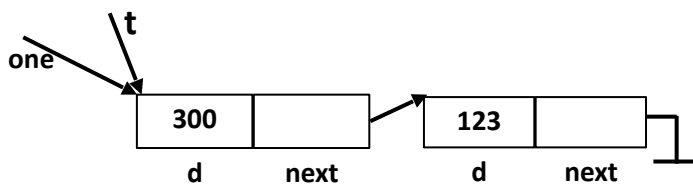
После просмотра элементов списка указатель t1 указывает на последний элемент списка.

Результат:

300 123

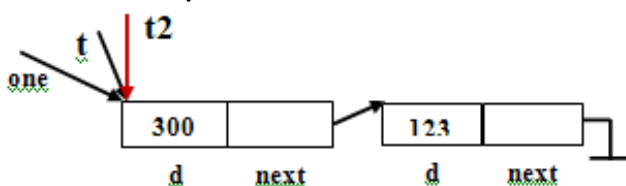
1.2. Вставка в конец списка

ВСТАВКА УЗЛА СО ЗНАЧЕНИЕМ 500 В КОНЕЦ СПИСКА



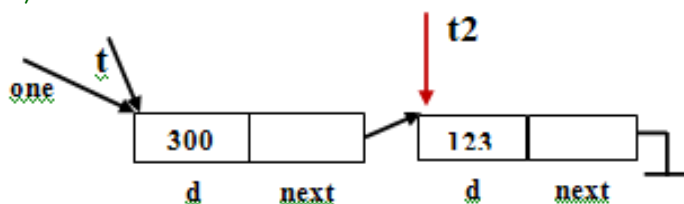
// 1 -----ПЕРЕВОД УКАЗАТЕЛЯ В КОНЕЦ СПИСКА -----

```
n *t2;
t2=one;
```



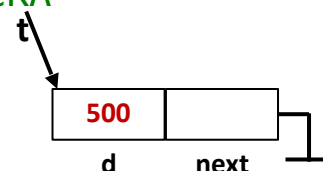
```
while ((*t2).next!=NULL) t2=(*t2).next;
```

```
/*
while (t2->next!=NULL) t2=t2->next;
*/
```



// 2 ----- СОЗДАНИЕ УЗЛА СПИСКА -----

```
t = new (n);
```




```

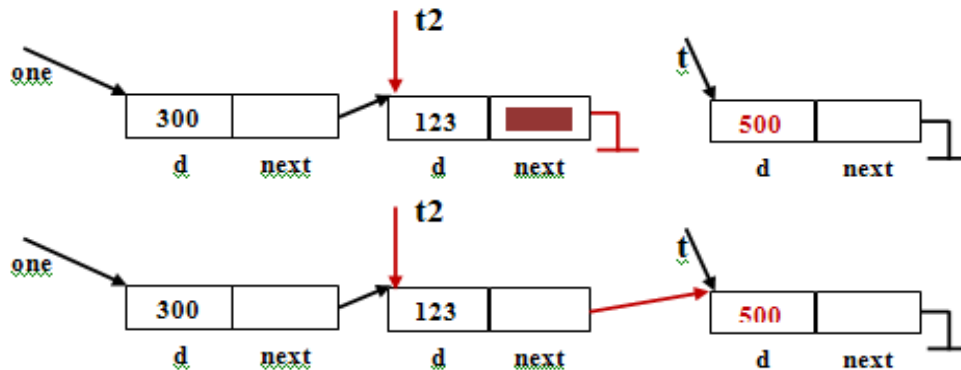
(*t).next=NULL;
(*t).d = 500;

```

```

// 3 ----- ПЕРЕВОД УКАЗАТЕЛЯ -----
//»присоединение» элемента 500 к исходному списку
(*t2).next=t;

```



Задание для выполнения в аудитории

Разработайте программу работы со списком (поле данных - **D**; указатель на следующий элемент списка - **Next**), в которой будут реализованы следующие действия над списком :

1. **инициализация** списка (создание первого элемента списка); 1 балл
2. **просмотр** элементов списка. Список выведите в следующем виде, например:

Адрес числа 530=0018A210; Адрес следующего эл-та списка= 0017C978

.....

Адрес числа 108=0018A210; Адрес следующего эл-та списка= 0x0

1 балл

3. **вставка** элементов списка (добавление элемента) с *заданной* информационной частью в *начало* списка. 1 балл
4. Организуйте ввод с клавиатуры:
 - количества элементов списка;
 - значения информационной части элемента списка;

1 балл

Задние для СРС 3 балла

Создайте список из N целочисленных элементов.

Требования к программе:

1. Организуйте ввод с клавиатуры количества элементов списка (N). 0,5 балла
 2. Значения информационной части элементов списка заполните «случайным» образом. 0,5 балла
 3. Вставка элементов в список – «вставка в конец списка». 0,5балла
 4. Выведите на экран список в процессе его создания. 0,5балла
 5. Вывод на экран элементов списка оформите как отдельную функцию. 1 балл
- Пример «передачи» списка в функцию:

```
void outsp(n* one)
{
// код вывода на экран (просмотра) списка
}
int main()
{
... ..
n *one; // указатель на первый элемент списка
outsp(one); // вызов функции вывода списка
... ..
}
```

В параметре функции «n *one»